

ECE 220 Computer Systems & Programming

Lecture 9 — Pointers and Arrays

September 22, 2026

Prof. Sayan Mitra `mitras@illinois.edu`

Course website: `courses.grainger.illinois.edu/ece220/fa2026`

I ILLINOIS

Electrical & Computer Engineering

GRAINGER COLLEGE OF ENGINEERING

The Address Problem

**You have been using addresses all along —
today they get a name.**

`scanf("%d", &n);` -- that & (Lecture 5)

swap couldn't reach main's x (Lecture 7)

`int *p;`

The Problem: swap by Value

```
/* Does this swap main's x and y? */
#include <stdio.h>

void swap(int a, int b) {
    int temp;
    temp = a;
    a = b;
    b = temp;
}

int main() {
    int x = 2, y = 3;
    swap(x, y);
    printf("x = %d, y = %d\n", x, y);
    return 0;
}
```

`main` passes `x` and `y` **by value**: `swap` receives *copies*, `a` and `b`, in its own activation record.

- Output? _____
- Why?

Draw the run-time stack at the end of `swap`:

To move `main`'s `x`, `swap` needs *where x lives* — its address. **Pointers, next.**

Declaring, &, and *

```
/* & takes an address; * follows one */
#include <stdio.h>
int main() {
    int variable = 4;
    int *ptr;           /* a pointer to int */
    ptr = &variable;    /* addr of variable */
    *ptr = *ptr + 1;     /* changes... what? */
    printf("variable = %d\n", variable);
    printf("ptr      = %p\n", (void *)ptr);
    return 0;
}
```

- `&variable` — the **address** of variable
- `*ptr` — the **value** at the address in ptr (dereference)

In-class: at the end,

variable = _____

ptr = _____

In-Class Problem: swap by Reference

```
void swap(int *a, int *b) {  
    /* declare a temporary */  
  
    /* exchange what a and b POINT TO */  
  
}  
  
int main() {  
    int x = 2, y = 3;  
    /* call swap with the ADDRESSES */  
  
    printf("x = %d, y = %d\n", x, y);  
    return 0;  
}
```

- The parameters become `int *a, int *b`
- The call becomes _____
- Output now: _____

Draw the run-time stack at the end of swap:

The Pointer Toolkit

- **Declare:** `type *p1;` or `type* p2;` — identical (the `*` binds to the *name*)
- **Create:** `type var;` `type *vp = &var;`
- **Dereference:** `*vp;` twice for a pointer to a pointer: `**vpp`
- **NULL:** a pointer that points at nothing — `ptr = NULL;` dereferencing it:

- `void *:` an address with no type — must be converted before dereferencing

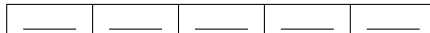
An uninitialized pointer points *somewhere* — writing through it is how programs corrupt themselves. Initialize to `NULL` or a real address.

Declaring and Using Arrays

```
int grid[5] = {10, 11, 12, 13, 14};
int i;
grid[4] = grid[1] + 1;
for (i = 0; i < 2; i++)
    grid[i+1] = grid[i] + 2;
for (i = 0; i < 5; i++)
    printf("%d ", grid[i]);
printf("\navg = %d\n", average(grid, 5));
```

- Five `ints`, *contiguous* in memory; indices run 0...4

In-class trace — final contents of `grid`:



Passing Arrays to Functions

```
int average(int array[], int size) { /* == int *  
    array */  
    int i, sum = 0;  
    for (i = 0; i < size; i++)  
        sum = sum + array[i];  
    return sum / size;  
}
```

Equivalent parameter spellings:

```
int array[10] = int array[] = int *array
```

- Arrays are passed **by reference**: what goes into the activation record is _____
- So the callee can *modify* the caller's array — no copying of 10 (or 10 million) elements
- That is also why the *size* must be passed separately

Pointer–Array Duality

```
char word[4]; char *cptr; cptr = word;
```

	pointer	ptr notation	array notation
	cptr	word	word[]
address of 1st element	cptr	word	_____
address of n th element	(cptr+n)	(word+n)	_____
value of 1st element	*cptr	*word	_____
value of n th element	*(cptr+n)	*(word+n)	_____

In-Class Problem: Reverse an Array

`array_reversal(a, n)`: first element becomes last, second becomes second-to-last, ...

- How many swaps for n elements?

- `array[i]`'s mirror is

- Why does `main` see the change?

```
void array_reversal(int array[], int n) {  
    /* declare a loop counter and a temporary */  
  
    /* how far does the loop need to go? */  
  
    /* swap array[i] with its mirror element */  
  
}
```

Input 1 2 3 4 5 6 → output _____

Summary & What's Next

Today

- **Pointers**: an address with a type; `&` takes an address, `*` follows one; NULL and uninitialized pointers are the sharp edges
- **swap works** when you pass addresses — still pass-by-value, but the value is an address
- **Arrays**: contiguous, 0-indexed, passed by reference (just the address of element 0) — and `a[n]` is `*(a+n)`

Next lecture

- Strings (arrays of char with a terminator) and multi-dimensional arrays

Code from today: `swap_value.c`, `pointers.c`, `swap_ptr.c`, `arrays.c`, `reverse.c`.